News-Driven Company Sentiment Pipeline

CSE 5114 — Data Manipulation and Management at Scale

Weikai Rao Kanan Ahmadov Abdou Sow Longhan Lin

***Abstract*—This paper presents a near-real-time data pipeline for tracking how companies are covered in the news and automatically scoring the sentiment of that coverage. Primary ingestion uses GDELT, with optional NewsAPI and RSS modes available when credentials, rate limits, or monitoring needs permit. Producer applications publish normalized events to the Kafka topic, Spark Structured Streaming consumes and enriches those events, and Snowflake stores both canonical sink tables and dashboard-ready reporting objects. Company mentions are identified through alias-based matching against curated dimension tables, and article sentiment is scored with VADER to produce positive, neutral, or negative labels. Snowflake dynamic tables then build a unified reporting layer that powers a Snowsight-based visualization artifact. The project demonstrates how a scalable data engineering pipeline can turn unstructured news events into structured sentiment and coverage signals that are useful for monitoring company reputation and media activity.**

I. Introduction

A. Motivation

News moves fast. A single article about a company can shift public perception, affect investor confidence, and in some cases move stock prices before any human analyst has had a chance to react. The challenge is that there is simply too much news to read manually. Thousands of articles are published every hour across hundreds of sources, and most of them do not come with any structured labels telling you which company they are about or whether the coverage is positive or negative.

This is the core problem we wanted to solve. If you are an investor, a risk manager, or even just someone trying to monitor how a company is being talked about in the media, you need a way to process news at scale and extract meaningful signals automatically. That kind of automation requires a well-designed data pipeline, not just a script that scrapes a few headlines.

B. Problem Statement

Raw news data is inherently unstructured. Articles do not have a company field. The same story gets picked up and republished by dozens of sources. Free-tier APIs limit how much data you can pull and give you only partial control over the time window you are querying. On top of that, the volume of data is large enough that any solution needs to be built with scalability in mind from the start.

The specific problems we set out to address were: (1) how do you ingest news articles in real time from multiple sources, (2) how do you figure out which company an article is actually talking about when there is no explicit label, (3) how do you score the sentiment of an article in a way that is fast enough to work in a streaming context, and (4) how do you store and aggregate all of this in a way that makes it easy to query and visualize.

C. Project Goals

Our goals for this project were to build an end-to-end pipeline that could do all of the above in a reproducible and scalable way. Specifically, we wanted to:

- Stream near-real-time news events primarily from GDELT, with optional NewsAPI and RSS ingestion modes, into a Kafka topic.
- Use Spark Structured Streaming as the processing engine to parse, normalize, deduplicate, match, and score articles.
- Store enriched results in Snowflake sink tables written directly by Spark.
- Build a reporting layer using Snowflake tables that refreshes automatically.
- Surface key metrics through a dashboard so that results are immediately accessible.

II. Data Sources

A. GDELT (Primary Source)

Our primary data source is GDELT, the Global Database of Events, Language, and Tone. GDELT monitors news media from around the world and makes its data publicly available, updating every 15 minutes. Each GDELT record includes fields such as the article URL, publication date, source country, and tone score. For our purposes, we primarily rely on the article URL, title, and publication timestamp for ingestion and deduplication, while other fields are treated as optional.

We chose GDELT as our primary source because it is freely available at scale, has broad international coverage, and updates frequently enough to support near-real-time use cases. It does not impose strict daily request quotas like commercial APIs, which makes it more practical for large-scale ingestion and testing. In practice, sending too many requests too quickly causes GDELT to return HTTP 429 errors, so our producer implements retry logic with exponential backoff to handle this.

B. NewsAPI and RSS (Supplementary)

In addition to GDELT, we set up producer applications for NewsAPI or RSS feeds as optional supplementary sources. NewsAPI is primarily used for prototye testing and targeted queries on specific companies, while RSS feeds provide curated domain-specific content. These sources help cover cases

that GDELT may underrepresent and allow us to supplement the GDELT baseline with more recent and focused articles.

The main limitation of NewsAPI is that the free tier restricts both the number of requests per day and the historical window you can query. In practice, this requires carefully scheduling and distributing requests over time to remain within quota while maintaining continuous data ingestion. As a result, NewsAPI is not suitable as a primary source but remains useful as a supplement during development and for testing the pipeline with a smaller, more controlled set of articles.

C. *Source Comparison and Limitations*

GDELT and NewsAPI serve different roles in our pipeline. GDELT gives us volume and breadth but the article content is often just a URL and metadata rather than full text. NewsAPI gives us more readable article content but is constrained by rate limits and time windows. Using both together means we can get reasonable coverage without being completely dependent on either one.

TABLE I: Comparison of GDELT and NewsAPI as data sources.

Property	GDELT	NewsAPI
Cost	Free	Free tier
Update frequency	Every 15 min	Plan-dependent freshness
Full article text	No (URL + metadata)	Partial (truncated content)
Rate limits	None	100 req/day
Historical window	Years	1 month
Coverage	Global	Mostly English

That said, both sources still have practical gaps for our use case. Although GDELT provides broad multilingual coverage, our collected sample can still underrepresent some regional or non-English coverage because of query design, source retrieval behavior, and downstream matching limitations. NewsAPI's free tier also limits the accessible historical window and request budget, making it less suitable for large-scale backfills. Neither source guarantees that every article will have a clean and usable title or description, which means our normalization step has to handle missing and malformed fields.

D. *Data Collection Challenges*

The main challenge with raw news data is the absence of a structured company field. Relevant articles may refer to a company using different names, ticker symbols, or implicit mentions, making simple filtering infeasible. Instead, relevance must be inferred from unstructured text using alias-based matching.

In addition, news data is often incomplete and inconsistent. Articles may lack titles or descriptions, and some sources provide only limited metadata without full text. Keyword-based retrieval further introduces noisy results, requiring additional normalization and filtering before downstream analysis.

III. Data Modeling

A. *Schema Design Overview*

Our Snowflake schema is organized into four layers, each serving a different purpose in the pipeline. The first layer is the raw data that Spark writes directly from the stream. The second is a set of canonical base tables that normalize and consolidate the raw writes. The third is a dynamic table reporting layer that automatically refreshes as new data arrives. The fourth is what the dashboard actually reads from.

This layered design was intentional. It keeps raw ingestion separate from reporting, which means we can iterate on the reporting logic without touching the ingestion pipeline. It also makes it easier to debug problems because we can inspect each layer independently.

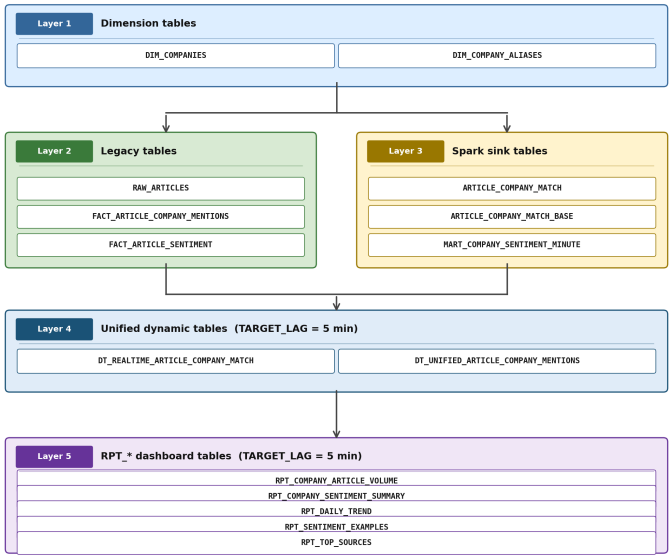

Fig. 1: Four-layer Snowflake schema design. Figure generated with assistance from Claude (Anthropic).

B. *Dimension Tables*

We maintain two main dimension tables. The first is `DIM_COMPANIES`, which stores the list of companies we track together with canonical identifiers such as ticker and company name. The second is `DIM_COMPANY_ALIASES`, which maps each company to a curated set of aliases used during mention matching.

Rather than deploying a heavier named entity recognition model inside the streaming path, we chose a simpler alias-based approach that is fast and interpretable. Expanding coverage is straightforward: adding a new company or improving matching behavior usually means inserting or revising rows in `DIM_COMPANY_ALIASES`.

C. *Fact and Sink Tables*

Spark writes enriched article records to three sink tables in Snowflake. `ARTICLE_COMPANY_MATCH` stores each article-company pair that was matched during the streaming job. `ARTICLE_COMPANY_MATCH_BASE` is the canonical version of this table, combining matches from both the legacy Airflow-based pipeline and the new Spark pipeline so that historical data and new data can be queried together. `MART_COMPANY_SENTIMENT_MINUTE` aggregates

sentiment counts and article volume by company and by minute.

D. *Dynamic Reporting Tables*

On top of the sink tables, we built a reporting layer using Snowflake dynamic tables. These objects refresh automatically with a configured target lag and keep the dashboard close to current data without requiring a separate scheduler for each reporting transform.

The key dynamic tables are `DT_REALTIME_ARTICLE_COMPANY_MATCH`, which deduplicates canonical real-time match rows, and `DT_UNIFIED_ARTICLE_COMPANY_MENTIONS`, which unifies legacy historical data with real-time data into a single reporting grain. On top of that unified layer, the `RPT_*` dynamic tables produce dashboard-facing summaries for article volume, sentiment summaries, daily trends, sentiment examples, and top news sources.

E. *Design Decisions and Tradeoffs*

The main design decision we debated was whether to use dynamic tables or scheduled tasks for the reporting layer. We chose dynamic tables because they are simpler to maintain and they eliminate the need to reason about scheduling intervals. The tradeoff is that dynamic tables in Snowflake can have some latency depending on how busy the warehouse is, but for our use case this was acceptable.

We also debated whether to write directly from Spark into the reporting tables or to keep a clean separation between the sink layer and the reporting layer. We went with the separation approach because it makes the pipeline more modular — if we want to change how sentiment is reported, we only need to update the dynamic table definitions, not the Spark job itself.

IV. Pipeline Architecture

A. *System Overview*

The pipeline has five main layers. News events are ingested primarily from GDELT, with optional NewsAPI and RSS modes available as supplementary sources. Producer applications publish normalized events to Kafka, Spark Structured Streaming consumes and enriches those events, Snowflake stores the sink tables, Snowflake dynamic tables build the reporting layer, and Snowsight reads the final `RPT_*` objects for visualization.

Figure 2 shows the full architecture diagram. The flow is top-down with clear boundaries between ingestion, streaming transformation, storage, and reporting. Although the stages are modular, the overall system is not stateless: Kafka offsets and Spark checkpoints provide recoverability, while Snowflake maintains the persistent sink and reporting layers.

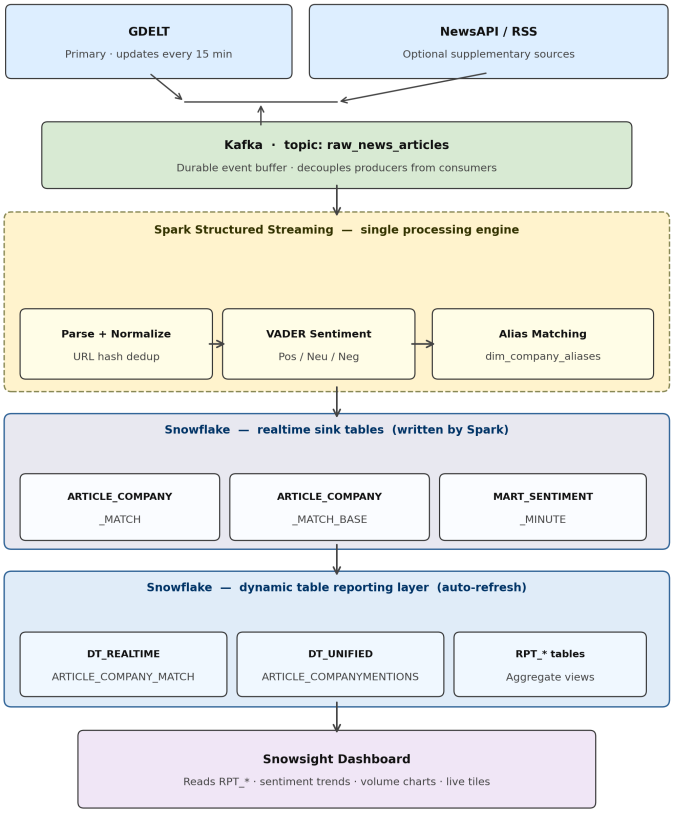

Fig. 2: System overview of the project pipeline. Figure generated with assistance from Claude (Anthropic).

B. *Ingestion Layer — Kafka*

All articles enter the pipeline through a single Kafka topic called `raw_news_articles`. Producer applications for GDELT and NewsAPI publish normalized article events to this topic. Each event is normalized into a JSON schema that includes fields such as the article URL, title, description, source name, and publication timestamp.

Using Kafka as the ingestion layer gives us a few important properties. First, it decouples the producers from the consumers, meaning we can add or remove sources without touching the Spark job. Second, it gives us a durable buffer so that if Spark falls behind or restarts, events are not lost. Third, it makes the pipeline naturally scalable because Kafka partitions can be consumed in parallel by multiple Spark executors.

C. *Processing Layer — Spark Structured Streaming*

Spark Structured Streaming is our single processing engine for all transformations. It handles everything in a single streaming job: parsing the raw Kafka events, normalizing fields, deduplicating by URL hash, loading the alias table from Snowflake, performing company matching, running VADER sentiment scoring, and writing the results to the sink tables. Each of these steps is implemented as a Spark transformation in a well-defined order, which makes the data lineage easy to follow.

D. Storage Layer — Snowflake

Spark writes to Snowflake using the Snowflake Spark connector, which handles batching and commit management automatically. The sink tables are append-only, meaning Spark never updates or deletes existing rows, only inserts new ones. This keeps the write logic simple and avoids conflicts.

Once data lands in the sink tables, Snowflake's dynamic tables take over. They are defined as SQL queries over the sink tables and refresh automatically as new rows arrive. The dashboard then reads from the outermost `RPT_*` tables, which means the end user is always seeing a clean, aggregated view rather than raw event data.

E. Scalability and Fault Tolerance

Kafka handles backpressure naturally. If Spark is processing slowly, events accumulate in the Kafka topic and are consumed when capacity is available. Spark's checkpointing mechanism ensures that if the streaming job crashes and restarts, it resumes from where it left off without reprocessing or losing events.

On the Snowflake side, dynamic tables refresh automatically according to their configured target lag, with Snowflake selecting incremental or full refresh behavior depending on query characteristics and support. This keeps the reporting layer manageable as the dataset grows without requiring us to schedule each downstream transform manually.

V. Data Processing

A. Parsing and Normalization

When a Kafka event arrives in Spark, the first step is to parse the JSON payload and extract the fields needed downstream, including URL, title, description, content, source name, provider metadata, and publication timestamp. We then standardize timestamps, normalize text fields, and build a combined `article_text` field by concatenating title, description, and content when available.

The current implementation drops records missing `url` or `title`, because those records cannot be reliably deduplicated or meaningfully matched and scored. URLs are lowercased and trimmed before hashing, and source/provider fields are normalized so they can be grouped consistently in downstream reporting.

B. Deduplication

We deduplicate articles by computing a SHA-256 hash of the normalized URL. This `url_hash` acts as a stable canonical key for downstream consolidation, especially when the same article is observed more than once during live ingestion.

The main limitation of URL-based deduplication is that it does not catch semantically identical stories published under different URLs. For the scope of this project, this tradeoff was acceptable because URL hashing is fast and easy to apply inside a streaming pipeline.

C. Company Matching via Alias Lookup

At the start of the Spark job, we load `DIM_COMPANY_ALIASES` from Snowflake and broadcast it so executors can match locally without repeated database calls. For each article, Spark checks whether the lowercased alias text appears within the lowercased `article_text` field and emits one row per matched article-company pair.

A single article can match multiple companies if multiple aliases appear. The main advantage of this design is operational simplicity: it is transparent, easy to debug, and easy to improve through alias curation. The main drawback is precision on ambiguous names, because the current implementation performs case-insensitive substring matching.

D. Sentiment Scoring with VADER

We chose VADER (Valence Aware Dictionary and sEntiment Reasoner) [1] for sentiment scoring because it is lightweight, requires no training, and runs fast enough to be used inside a Spark streaming job without significant overhead. VADER works by looking up words in a hand-crafted sentiment lexicon and combining their scores into a single compound score between -1 and +1.

We apply VADER [1] to the concatenation of the article title and description. The resulting compound score is then classified using fixed thresholds: articles with a compound score above 0.05 are labeled positive, below -0.05 are labeled negative, and everything in between is neutral. These thresholds are the standard VADER recommendations.

The main limitation of VADER for this use case is that it was designed for social media text and does not have specific knowledge of financial language. Words like "bearish" or "downgrade" that carry clear negative connotations in a financial context might not be scored correctly by VADER. This is something we would address in future work by switching to a finance-tuned model like FinBERT.

E. Data Quality Checks

At each stage of the pipeline we apply basic quality checks. Records missing `url` or `title` are dropped during normalization. Records that fail URL hashing are logged and skipped. Company matches that produce an unusually high number of matches for a single article (more than ten companies) are flagged as potential false positives and reviewed separately. Sentiment scores that are exactly zero across all VADER subcategories are flagged as likely scoring failures.

These checks are intentionally lightweight. The goal is to catch obvious data quality issues without adding latency to the streaming job.

VI. Results and Analysis

A. Sentiment Trends by Company

The reporting layer allows us to compare sentiment across tracked companies using both aggregate and example-level views. The dashboard surfaces stacked sentiment counts and average sentiment by company, making it easy to see which

companies are receiving mostly positive, neutral, or negative coverage during the observation window.

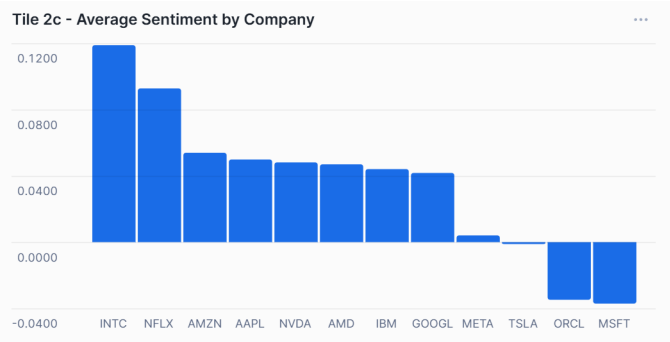

Fig. 3: Average sentiment score per company.

In practice, the average-sentiment view is most informative when interpreted together with article counts. Companies with only a small number of matched articles can show larger swings from a single strongly worded article, while heavily covered companies tend to have more stable averages.

B. *News Volume Patterns*

Our final dashboard emphasizes an aggregate company-level sentiment view rather than a daily trend view. Figure 4 summarizes the full observation window by showing, for each tracked company, the number of matched articles partitioned into negative, neutral, and positive sentiment classes.

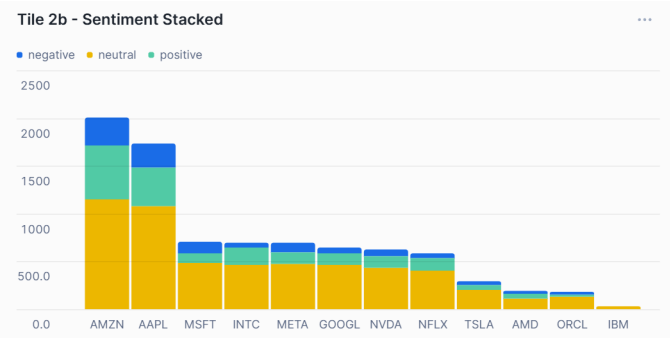

Fig. 4: Article volume per company broken down by sentiment label.

The chart highlights substantial differences in media attention across companies. Amazon and Apple account for the largest volumes of matched coverage, while several other companies appear at much lower frequencies. Across nearly all companies, neutral articles form the largest share of coverage, which is consistent with the fact that many news articles are descriptive and informational rather than strongly opinionated.

Even though this figure does not show temporal variation, it still serves an important analytical purpose. It allows us to compare both total coverage volume and sentiment composition in one visualization. Higher overall bars indicate greater media attention, while the balance among negative, neutral, and positive segments helps reveal whether that attention is comparatively favorable, unfavorable, or mostly neutral over the full collected dataset.

C. *Matching Coverage and Accuracy*

We evaluate matching quality primarily through dashboard diagnostics and manual spot checks. The matching-quality tile compares total ingested articles with matched and unmatched articles, which gives a direct view of coverage at the article grain. Manual inspection of example rows suggests that the alias-based method performs reasonably well for distinctive company names but is weaker for ambiguous and short aliases.

Tile 6 - Matching Quality 1 row

TOTAL_ARTICLES	MATCHED_ARTICLES	UNMATCHED_ARTICLES
10842	7751	3091

Fig. 5: The matching quality of total articles collected.

This is an expected tradeoff of the current design. The implementation favors speed, interpretability, and ease of extension, while accepting that some false positives and false negatives remain. That tradeoff was appropriate for a working end-to-end class project, though it leaves room for stronger entity resolution in future work.

VII. Limitations

A. *VADER Limitations for Financial Text*

VADER [1] was not designed for financial news. It was originally built for short social media text, which means it lacks awareness of financial jargon. In our pipeline, this shows up in cases where an article uses phrases like “missed earnings,” “downgraded,” or “bearish outlook” (terms that carry a clearly negative signal to any finance reader) but receive a neutral or even slightly positive VADER compound score because the surrounding sentence structure is not overtly negative. Since our classification thresholds follow the standard VADER recommendations (compound ≥ 0.05 for positive, ≤ -0.05 for negative), these misclassifications flow directly into our `RPT_COMPANY_SENTIMENT_SUMMARY` aggregates and could mislead a downstream consumer of the data. The most practical fix would be replacing VADER with a finance-tuned model such as FinBERT [2], which was pre-trained on financial news and handles this vocabulary more appropriately.

B. *Alias Matching Edge Cases*

Our alias-based matching approach, implemented as a broadcast lookup inside Spark, is fast and easy to maintain, but it has well-known precision issues with short or ambiguous company names. The clearest example in our seed data is Apple: the alias `Apple` can match any article mentioning Apple products, apple harvests, or the fruit in passing. At present, the implementation primarily relies on case-insensitive substring matching, which keeps the pipeline simple and fast but leaves precision issues for names like `Meta`, `AMD`, or `IBM`,

which are short and can appear in non-financial contexts. A named entity recognition model operating on full article text would resolve these ambiguities more reliably, at the cost of higher per-record processing time inside the Spark streaming job.

C. *Data Coverage Gaps*

Although GDELT provides broad multilingual coverage, our collected sample can still underrepresent some regional and non-English coverage because of query design, source retrieval behavior, and downstream matching limitations. For the twelve companies currently seeded in `DIM_COMPANIES` (all US-listed NASDAQ and NYSE equities), this issue is less severe, but it would become more important if we expanded tracking to a larger international company set. A second coverage gap is temporal: GDELT updates every 15 minutes, so our pipeline has an inherent minimum ingestion lag on that order even before downstream processing and reporting refreshes are considered. For most analytical use cases this is acceptable, but it rules out sub-minute latency applications.

VIII. Conclusion and Next Steps

A. *Summary of Contributions*

This project delivers a working, end-to-end near-real-time pipeline for tracking company news sentiment.

The core engineering contribution is not a novel algorithm but rather the design of a modular and reproducible pipeline where each layer has a single responsibility and can be inspected, tested, and replaced independently. The separation between the Spark sink layer and the Snowflake dynamic table reporting layer in particular proved valuable during development: we were able to iterate on the SQL definitions and the dashboard tile queries without touching the Spark job at all.

B. *Future Work*

The most impactful improvement would be replacing VADER with a finance-tuned sentiment model. FinBERT is the natural candidate, but integrating a transformer model into a Spark Structured Streaming job requires either broadcasting model weights to each executor or running inference through a separate microservice that Spark calls per micro-batch. Either approach adds engineering complexity but would meaningfully improve the quality of the aggregates that the dashboard currently surfaces.

Finally, we would like to extend the tracked company universe beyond the current twelve US companies, set up a longer historical GDELT backfill, and use the resulting dataset to run a more rigorous offline evaluation of both the matching precision and the sentiment quality before relying on the pipeline for any downstream modeling.

References

[1] C. Hutto and E. Gilbert, "Vader: A parsimonious rule-based model for sentiment analysis of social media text," *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 8, no. 1, pp. 216–225, May 2014. [Online]. Available: https://ojs.aaai.org/index.php/ICWSM/article/view/14550

[2] D. Araci, "Finbert: Financial sentiment analysis with pre-trained language models," *CoRR*, vol. abs/1908.10063, 2019. [Online]. Available: http://arxiv.org/abs/1908.10063